Ensemble Learning and Random Forest

Lecturer: Albert Yang (楊景明)

Ensembel Learning

The Problem of Decision Tree

- Decision Trees are easy to use, easy to build, and easy to interpret

The Problem of Decision Tree

- Decision Trees are easy to use, easy to build, and easy to interpret
- However, “Trees have one aspect that prevents them from being the ideal tool for predictive learning, namely **inaccuracy**”

The Problem of Decision Tree

- Decision Trees are easy to use, easy to build, and easy to interpret
- However, “Trees have one aspect that prevents them from being the ideal tool for predictive learning, namely **inaccuracy**”
- That is, they work great with the data used to create them, but they are not flexible, when it comes to classifying **new samples**

The Problem of Decision Tree

- Decision Trees are easy to use, easy to build, and easy to interpret
- However, “Trees have one aspect that prevents them from being the ideal tool for predictive learning, namely **inaccuracy**”
- That is, they work great with the data used to create them, but they are not flexible, when it comes to classifying **new samples**
- **Random forest** combines the simplicity of decision tree with flexibility resulting in a vast improvement in accuracy

The Problem of Decision Tree

- Decision Trees are easy to use, easy to build, and easy to interpret
- However, “Trees have one aspect that prevents them from being the ideal tool for predictive learning, namely **inaccuracy**”
- That is, they work great with the data used to create them, but they are not flexible, when it comes to classifying **new samples**
- **Random forest** combines the simplicity of decision tree with flexibility resulting in a vast improvement in accuracy
- Random Forest is an **ensemble learning method**

Ensemble Learning

- If you aggregate the predictions of a group of predictors, you will often get better predictions than with the best individual predictor
- A group of predictors is called an **ensemble**; this technique is called **Ensemble Learning**

Ensemble Learning

- For example, you can train a group of **Decision Tree** classifiers, each on a different random subset of the training set (**Bagging**)
- To make predictions, you just obtain the predictions of all individual trees, then predict the class that gets the **most votes**
- Such an ensemble of Decision Trees is called a **Random Forest**
- Despite its simplicity, this is one of the most powerful Machine Learning algorithms available today

Voting Classifiers

- Suppose you have trained a few classifiers, each one achieving about **80**% accuracy. You may have a **Logistic Regression** classifier, an **SVM** classifier, a **Random Forest** classifier, a **K-Nearest Neighbors** classifier, and perhaps a few more

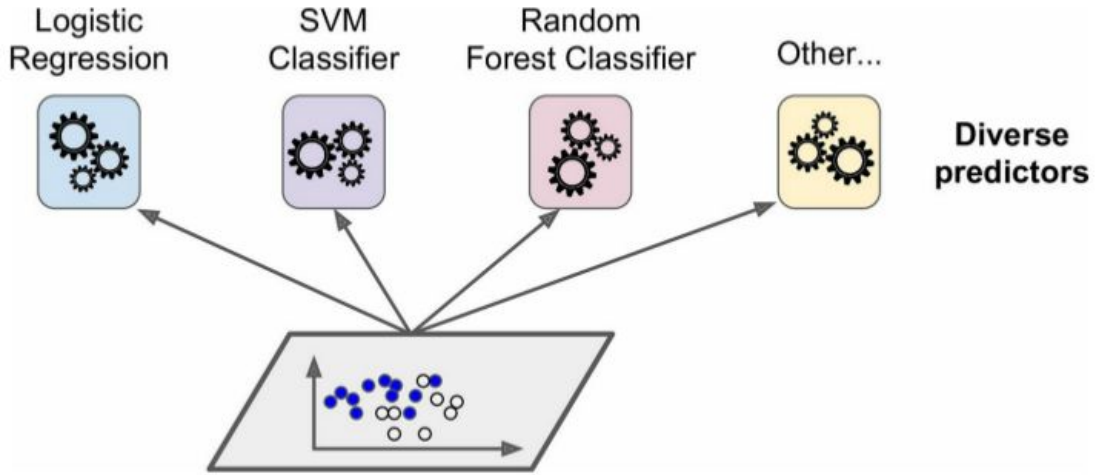


Figure 7-1. Training diverse classifiers

Voting Classifiers

- A very simple way to create an even better classifier is to **aggregate** the predictions of each classifier and predict the class that gets the **most votes**
- This majority-vote classifier is called a **hard voting** classifier

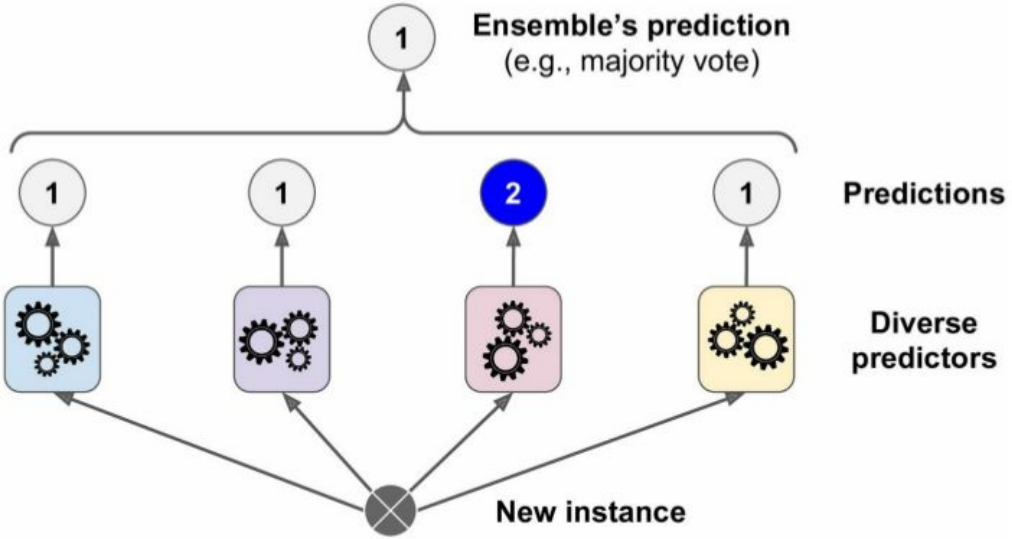


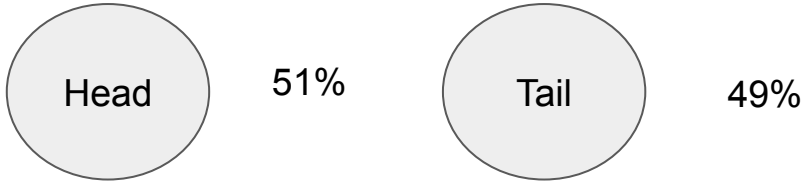
Figure 7-2. Hard voting classifier predictions

Two Heads are Better Than One

- This voting classifier often achieves a higher accuracy than the best classifier in the ensemble
- In fact, even if each classifier is a **weak learner** (meaning it does only slightly better than random guessing), the ensemble can still be a **strong learner** (achieving high accuracy)
- As long as there are a **sufficient number** of weak learners and they are **sufficiently diverse**

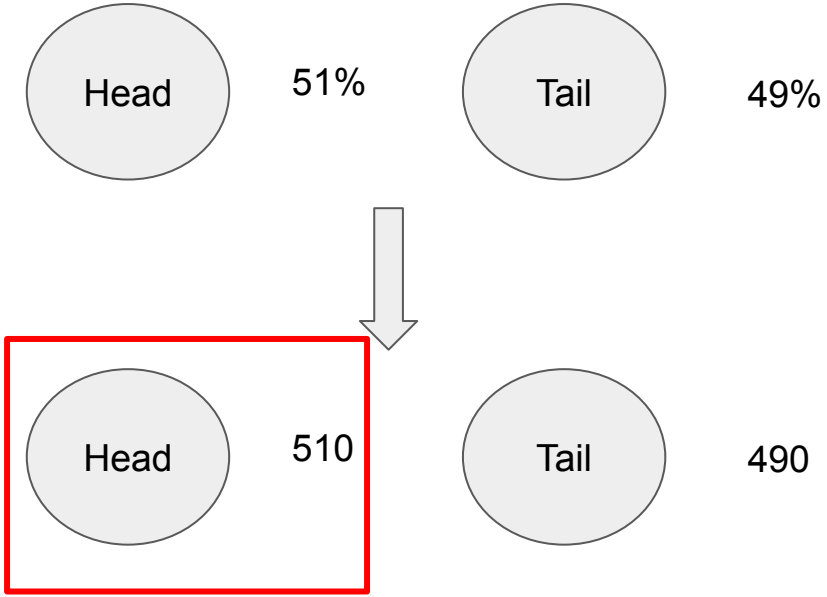
Why

Suppose you have a slightly biased coin…



Why

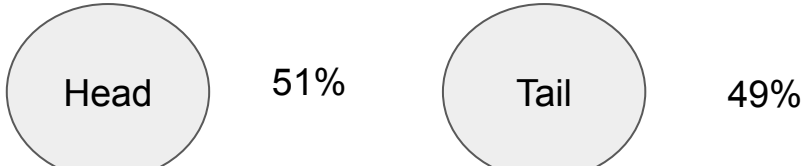
Suppose you have a slightly biased coin…



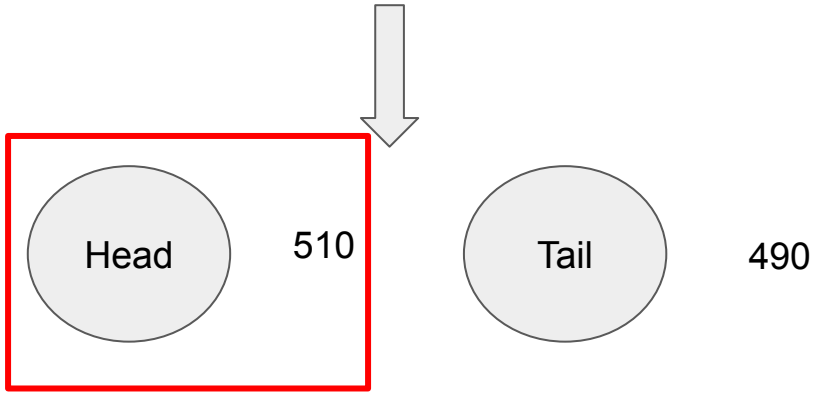
After 1,000 tosses

Why

Suppose you have a slightly biased coin…



After 1,000 tosses



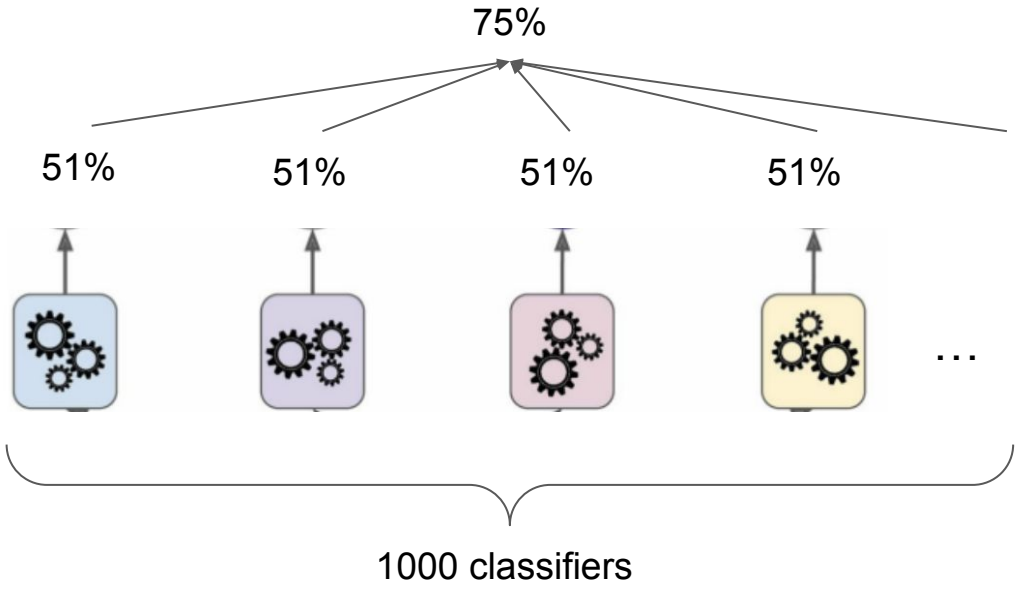
- If you repeat this more times
- The probability of obtaining a **majority** (>500) of heads after 1,000 tosses is close to 75%

The Law of Large Numbers

Why 75%? (extra)

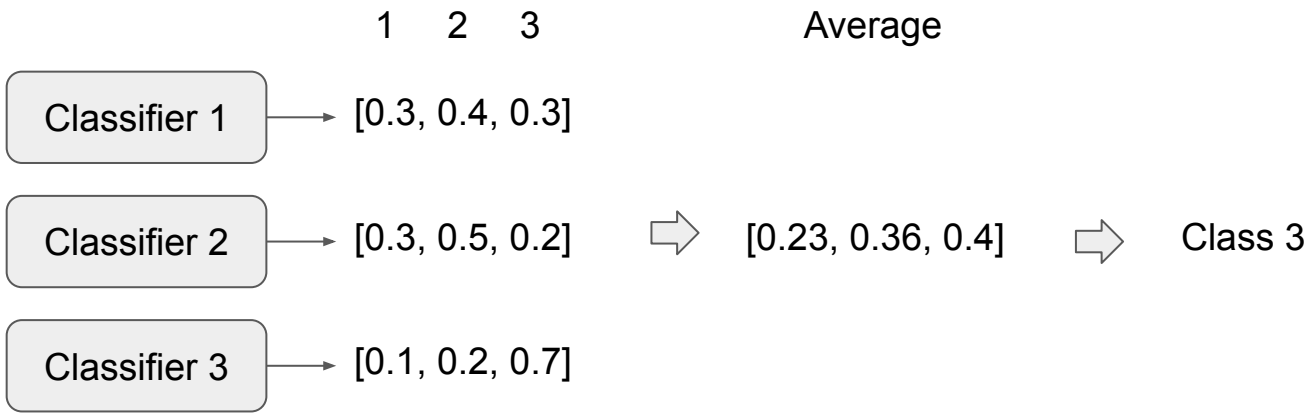
- According to **Central Limit Theorem** (CLT) at N=1,000
  - The **Binomial Distribution** (p=0.51) is approximated by the Normal Distribution
  - The **Mean (μ)** is 510 (1,000×0.51)
- Calculating the Probability P(Head>500)
  - We are seeking the area of the Normal Curve to the right of 500
- Step 1: calculate the Z score
  - In our case (N=1000), mean = 510, SD = 15.8082
  - $Z = (X_{threshold} - mean) / SD \approx (500 - 510) / 15.8082 \approx -0.63$
- Step 2: calculate the Probability P(Head>500)
  - Probability $P(Head>500) = P(Z \geq -0.63) = 1 - P(Z < -0.63) \approx 1 - 0.2643 \approx 0.74$

- Similarly, suppose you build an ensemble containing **1,000** classifiers that are individually correct only **51**% of the time (barely better than **random guessing**)
- If you predict the majority voted class, you can hope for up to **75**% accuracy!
- However, this is only true if all classifiers are perfectly **independent**, making uncorrelated errors, which is clearly not the case since they are trained on the same data
- They are likely to make the same types of errors, so there will be many majority votes for the wrong class, reducing the ensemble's accuracy



Soft Voting

- Another voting method is to average the class probabilities of all individual classifiers and then predict the class with the highest class probability, called **soft voting**
- Soft voting often achieves higher performance than hard voting because it gives more weight to highly **confident votes**



Bagging and Pasting

- In ensemble learning, having **independent classifiers** is crucial
- One way to get a diverse set of classifiers is to use **very different training algorithms**
- Another approach is to use the **same training algorithm** for every predictor, but to train them on **different random subsets** of the training set

Bagging and Pasting

- When sampling is performed with **replacement**, this method is called **bagging** (short for bootstrap aggregating)
- When sampling is performed **without replacement**, it is called **pasting**

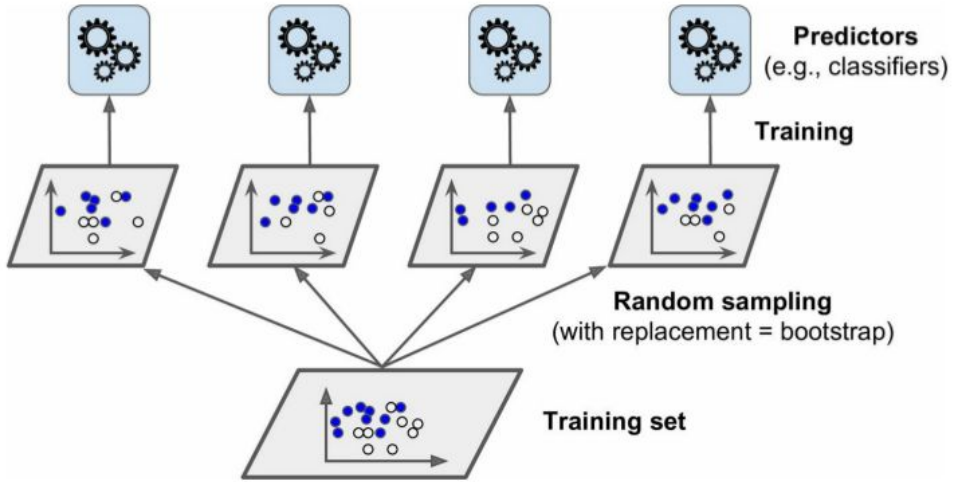


Figure 7-4. Pasting/bagging training set sampling and training

Bagging and Pasting

- Once all predictors are trained, the ensemble can make a prediction for a new instance by simply aggregating the predictions of all predictors
- Classification -> **majority**; Regression -> **average value**

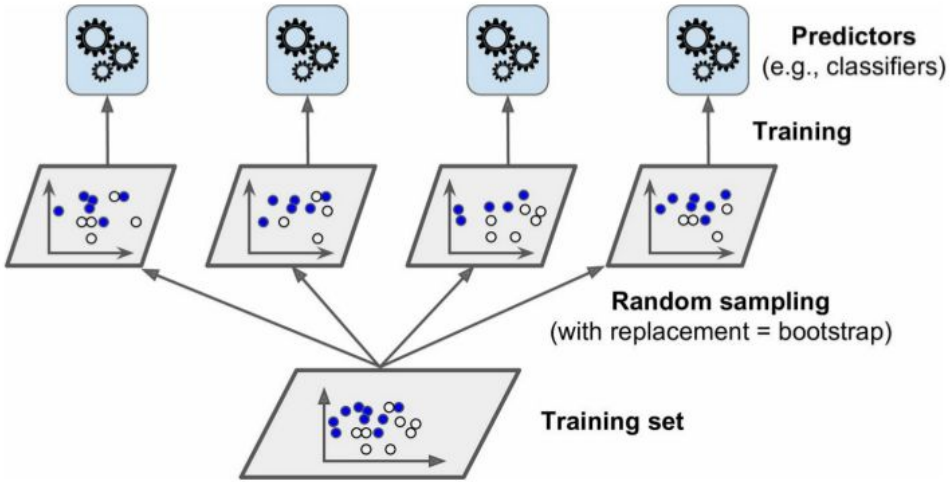
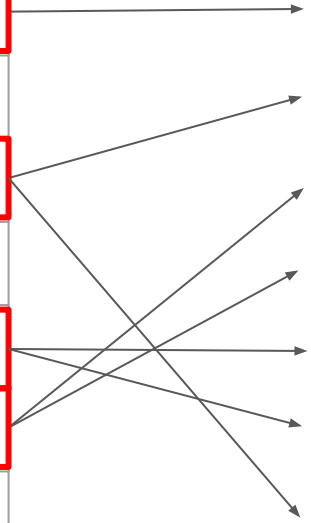


Figure 7-4. Pasting/bagging training set sampling and training

Example: Bagging

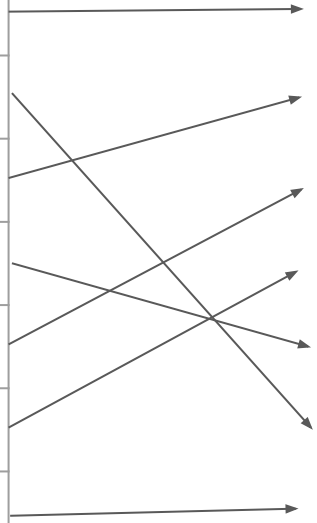
Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
No	Yes	18	Yes
Yes	No	50	No
Yes	No	50	No
Yes	Yes	38	Yes
Yes	Yes	38	Yes
No	Yes	18	Yes

Example: Pasting

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
No	Yes	18	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	Yes	35	Yes
Yes	No	12	No
No	No	83	No

Lower Variance

- Each individual predictor has a higher bias than if it were trained on the original training set
- But aggregation **reduces both bias and variance**
- Generally, the net result is that the ensemble has a similar bias but a **lower variance** than a single predictor trained on the original training set

Scalability

- Predictors can all be trained in parallel
- Predictions can be made in parallel
- Scale very well!

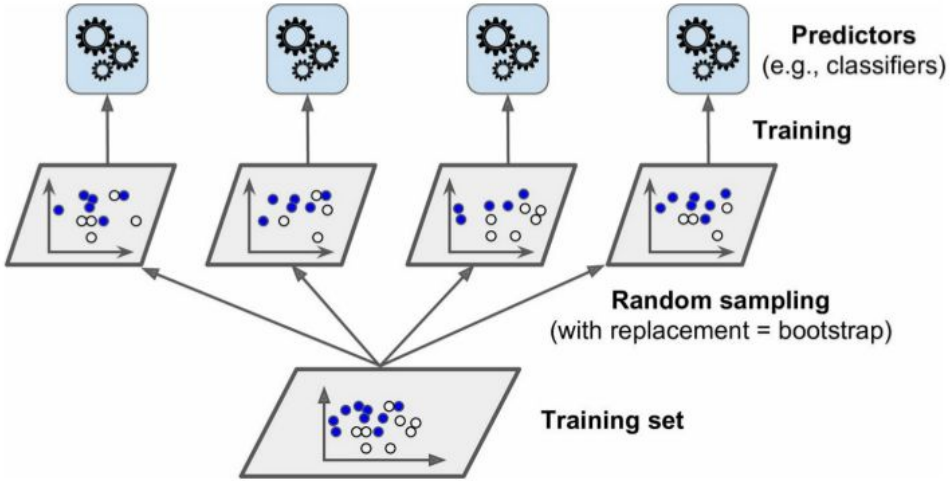


Figure 7-4. Pasting/bagging training set sampling and training

Bagging vs Pasting

- **Bootstrapping (or bagging)** introduces a bit **more diversity** in the subsets that each predictor is trained on, so bagging ends up with a slightly **higher bias** than pasting
- But this also means that predictors end up being less correlated so the **ensemble's variance is reduced**
- Overall, **bagging often results in better models**, which explained why it is generally preferred

Out-of-Bag Evaluation

- With bagging, some instances may be sampled several times for any given predictor, while others may not be sampled at all
- By default a BaggingClassifier samples **m** training instances with replacement (bootstrap=True), where **m** is the size of the training set.
- In general, only about **63**% of the training instances are sampled on average for each predictor.
- The remaining **37**% of the training instances that are not sampled are called **out-of-bag** (oob) instances.
- Note that they are not the same **37**% for all predictors.

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
No	Yes	18	Yes
Yes	No	50	No
Yes	No	50	No
Yes	Yes	38	Yes
Yes	Yes	38	Yes
No	Yes	18	Yes

Out-of-bag instances

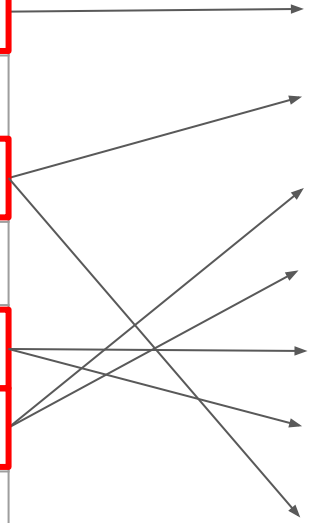
- Since a predictor never sees the oob instances during training, it can be evaluated on these instances, **without the need for a separate validation set or cross-validation**
- You can evaluate the ensemble itself by **averaging out the oob evaluations of each predictor**

Random Patches and Random Subspaces

- In addition to instance sampling, we can also do **feature sampling**. Thus, each predictor will be trained on a random subset of the input features
- This is particularly useful when you are dealing with **high-dimensional inputs** (such as images)
- Sampling both training instances and features is called the **Random Patches** method
- Keeping all training instances but sampling features is called the **Random Subspaces** method
- Sampling features results in even more predictor diversity, **trading a bit more bias for a lower variance**

Example: Random Patches

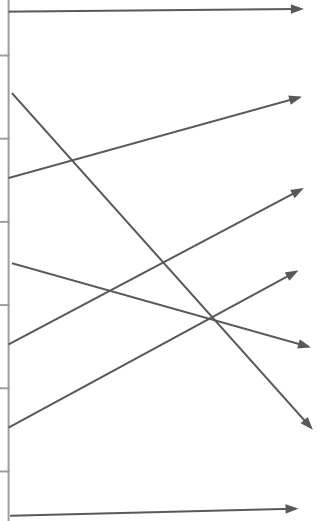
Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



Loves Popcorn	Loves Soda	Loves Titanic
Yes	Yes	No
No	Yes	Yes
Yes	No	No
Yes	No	No
Yes	Yes	Yes
Yes	Yes	Yes
No	Yes	Yes

Example: Random Subspaces

Loves Popcorn	Loves Soda	Age	Loves Titanic
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

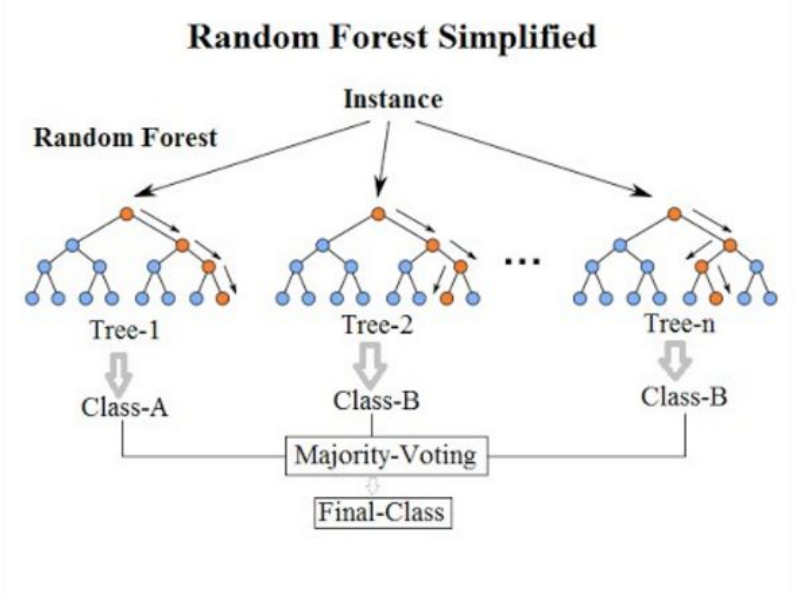


Loves Popcorn	Loves Soda	Loves Titanic
Yes	Yes	No
No	Yes	Yes
Yes	Yes	Yes
Yes	No	No
No	Yes	Yes
Yes	No	No
No	No	No

Random Forest

Random Forest

- **Random Forest** is an ensemble of Decision Trees
- Generally trained via the **bagging** method (or sometimes pasting)
- Random Forest = Bagging + Decision Trees



Steps to Build Random Forest

- Step 1: Defining a random sample of size **n** (bagging or pasting)
- Step 2: From the selected n data, a **decision tree** is trained. For each tree:
  - a. Randomly select **d** features
  - b. Use features to split the node (e.g. calculate **Gini Impurity**)
- Step 3: Repeat steps 1 to 2
- Step 4: The predictions of all decision trees are aggregated, and the classification result is determined by **majority rule**

Boosting

Boosting

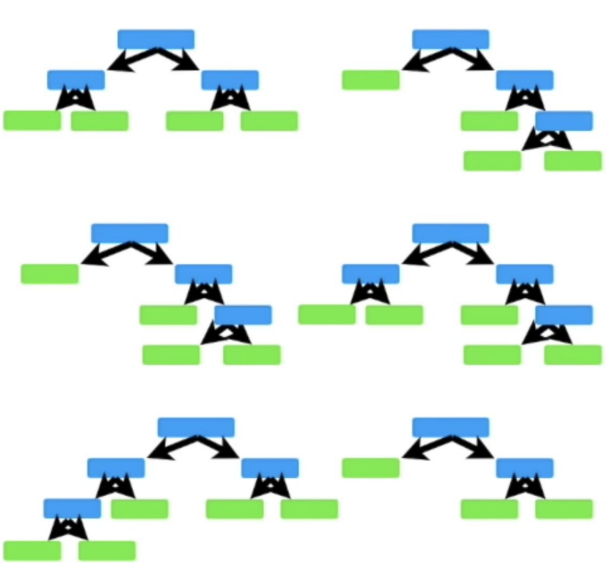
- **Boosting** refers to any ensemble method that can combine several weak learners into a strong learner
- The general idea of most boosting methods is to train predictors **sequentially**, each trying to correct its predecessor
- By far the most popular are **AdaBoost** (short for Adaptive Boosting) and **Gradient Boosting**

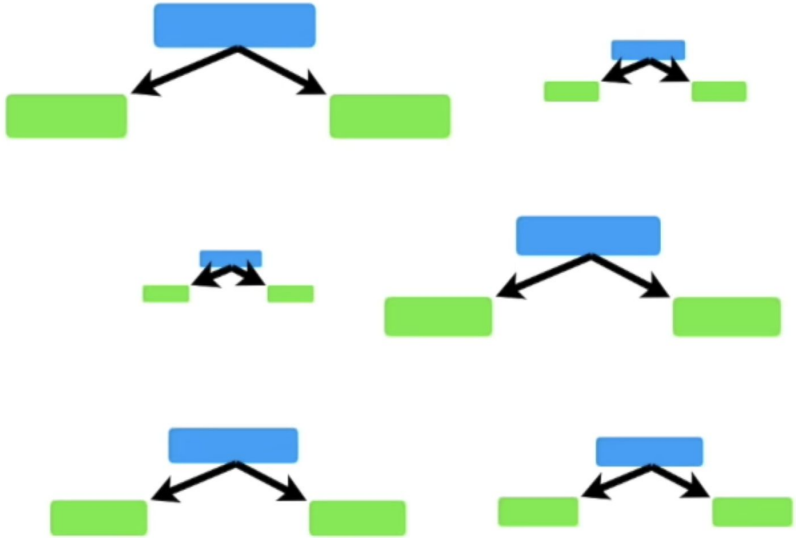
AdaBoost

- One way for a new predictor to correct its predecessor is to **pay a bit more attention to the training instances that the predecessor underfitted**
- This results in new predictors focusing more and more on the hard cases. This is the technique used by **AdaBoost**

Random Forest vs AdaBoost

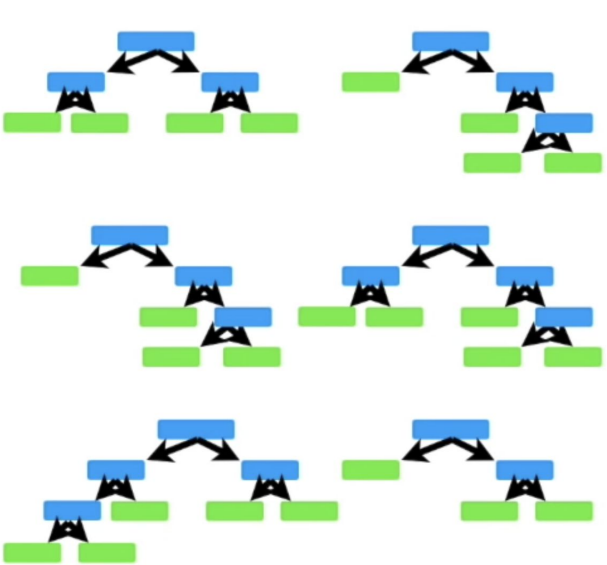
- Random Forest consists of many Decision Trees with **different depth**
- AdaBoost is composed by many **stumps** (Trees with depth = 1)

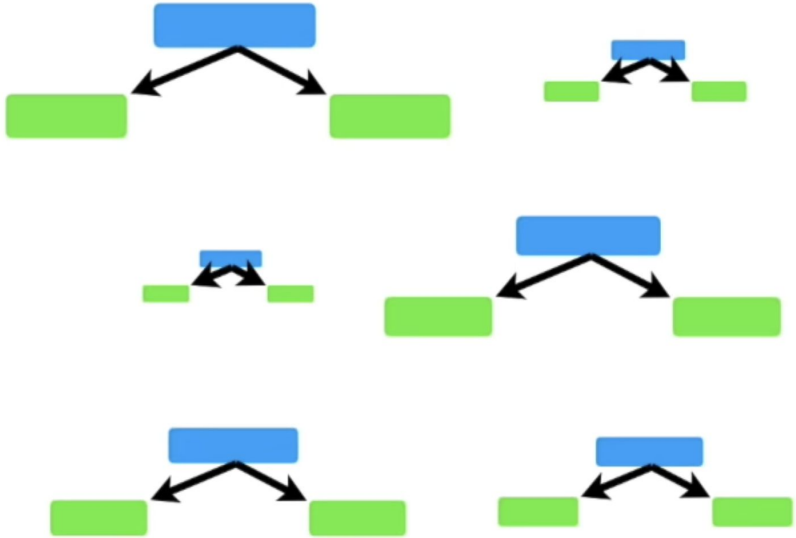




Random Forest vs AdaBoost

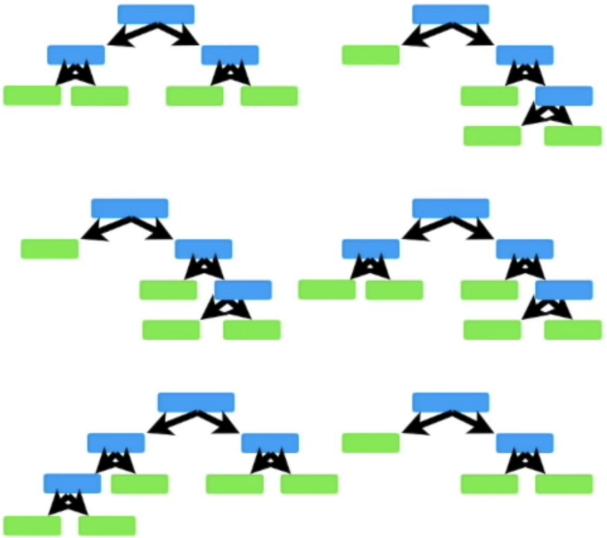
- In Random Forest, each tree has an equal vote on the final classification
- In AdaBoost, some stumps get more say in the final classification than others

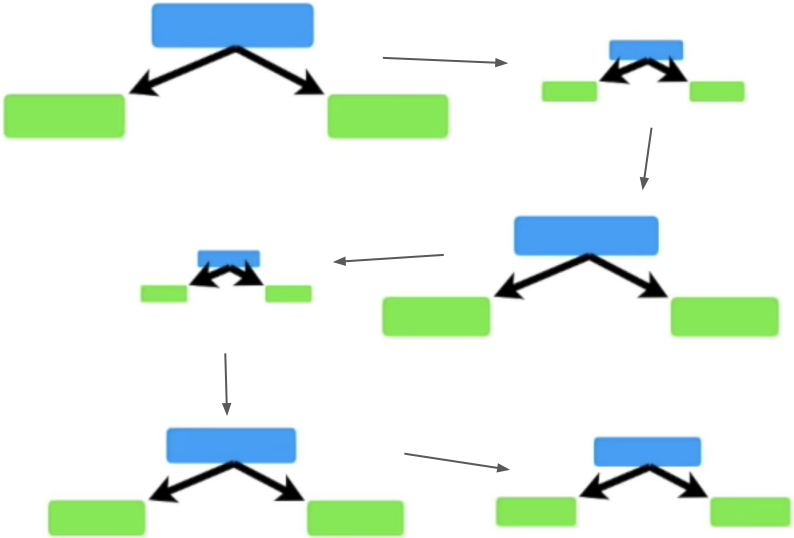




Random Forest vs AdaBoost

- In Random Forest, trees are trained simultaneously
- In AdaBoost, trees are trained sequentially





AdaBoost

- For example, to build an AdaBoost classifier, a first base classifier (such as a Decision Tree) is trained and used to make predictions on the training set

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8

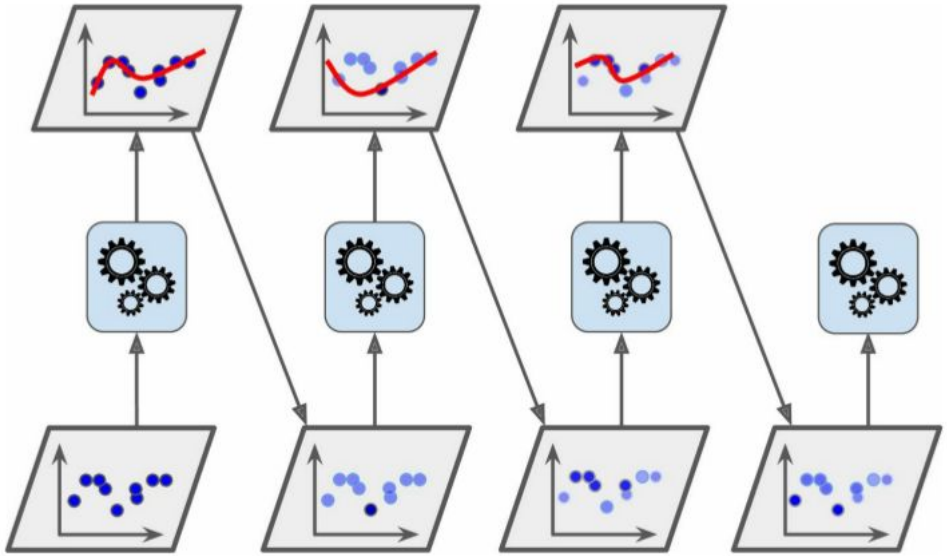


Figure 7-7. AdaBoost sequential training with instance weight updates

How to update **sample weights**: https://www.youtube.com/watch?v=LsK-xG1cLYA

AdaBoost

- **The relative weight of misclassified training instances is then increased**
- A second classifier is trained using the updated weights and again it makes predictions on the training set, weights are updated, and so on

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	0.07
No	Yes	180	Yes	0.07
Yes	No	210	Yes	0.07
Yes	Yes	167	Yes	0.49
No	Yes	156	No	0.07
No	Yes	125	No	0.07
Yes	No	168	No	0.07
Yes	Yes	172	No	0.07

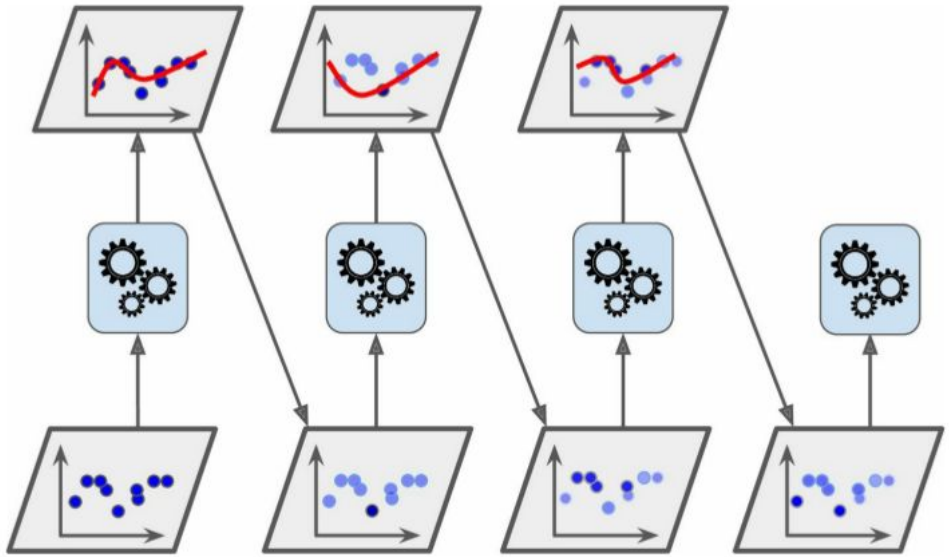


Figure 7-7. AdaBoost sequential training with instance weight updates

Drawback

- There is one important drawback to this sequential learning technique: **it can't be parallelized**
- Each predictor can only be trained after the previous predictor has been trained and evaluated
- It doesn't scale as well as bagging or pasting

Gradient Boosting

- Just like AdaBoost, **Gradient Boosting** works by sequentially adding predictors to an ensemble, each one correcting its predecessor.
- However, instead of tweaking the instance weights at every iteration like AdaBoost does, this method tries to **fit the new predictor to the residual errors made by the previous predictor**

Step 1: Make the First Guess

X1	X2	X3	X4	Y
1.20	14.80	0.40	28.50	14.10
1.30	18.40	0.38	28.60	9.87
2.05	11.40	0.43	23.50	11.50
1.80	9.80	0.67	19.90	18.10



X1	X2	X3	X4	Y	Avg_Pred
1.20	14.80	0.40	28.50	14.10	13.39
1.30	18.40	0.38	28.60	9.87	13.39
2.05	11.40	0.43	23.50	11.50	13.39
1.80	9.80	0.67	19.90	18.10	13.39

How to make the first guess

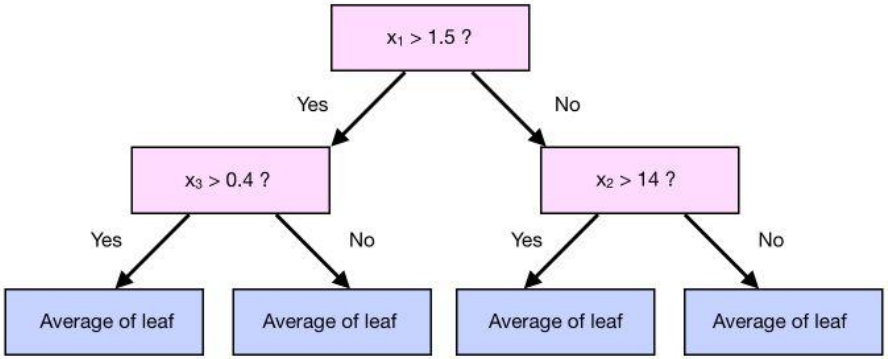
- Average value
- Use a simple Tree Regressor

Step 2: Compute the Pseudo-Residuals

X1	X2	X3	X4	Y	Avg_Pred	Pseudo_Res
1.20	14.80	0.40	28.50	14.10	13.39	0.71
1.30	18.40	0.38	28.60	9.87	13.39	-3.52
2.05	11.40	0.43	23.50	11.50	13.39	-1.89
1.80	9.80	0.67	19.90	18.10	13.39	4.71

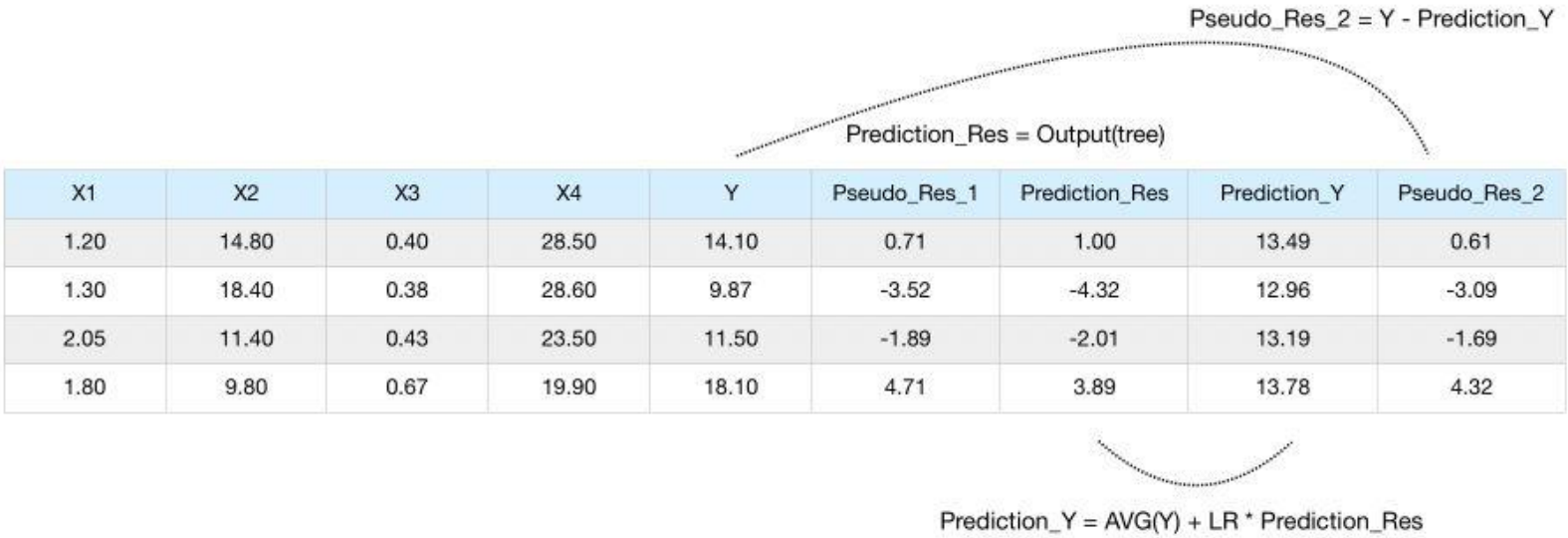
- We compute the difference between the observations and the prediction we made
- This is called the **pseudo-residuals**

Step3: Predict the Pseudo-Residuals



- Fit a regression tree to predict the **pseudo-residuals**
- Limit the number of leaves (e.g., 8–32), so there are **fewer leaves than samples**
- Each leaf outputs the **average residual** of the samples that fall into it

Step 4: Make a Prediction and Compute the Residuals



X1	X2	X3	X4	Y	Pseudo_Res_1	Prediction_Res	Prediction_Y	Pseudo_Res_2
1.20	14.80	0.40	28.50	14.10	0.71	1.00	13.49	0.61
1.30	18.40	0.38	28.60	9.87	-3.52	-4.32	12.96	-3.09
2.05	11.40	0.43	23.50	11.50	-1.89	-2.01	13.19	-1.69
1.80	9.80	0.67	19.90	18.10	4.71	3.89	13.78	4.32

- To avoid overvitting, Gradient Boost applies a learning rate *lr* to **scale the contribution from a new tree**, by applying a factor between 0 and 1
- The idea behind the learning rate is to **make a small step in the right direction**
- This allows an **overall lower variance**

Step 5: Make a Second Prediction

$$y_{pred} = \text{AVG(Y)} + lr \times res_{pred_1} + lr \times res_{pred_2}$$

- Build a second tree
- Compute the prediction using this second tree
- Compute the residuals according to the prediction
- Build the third tree
- …
- We reach the maximum number of trees specified or we don’t learn significantly anymore

More details: https://www.youtube.com/watch?v=3CC4N4z3GJc